

Acme Financial Data Warehouse — Project Report

Author: Lazea Cristian
Course: Data Warehouse
Date: May 2026

1. Executive Summary

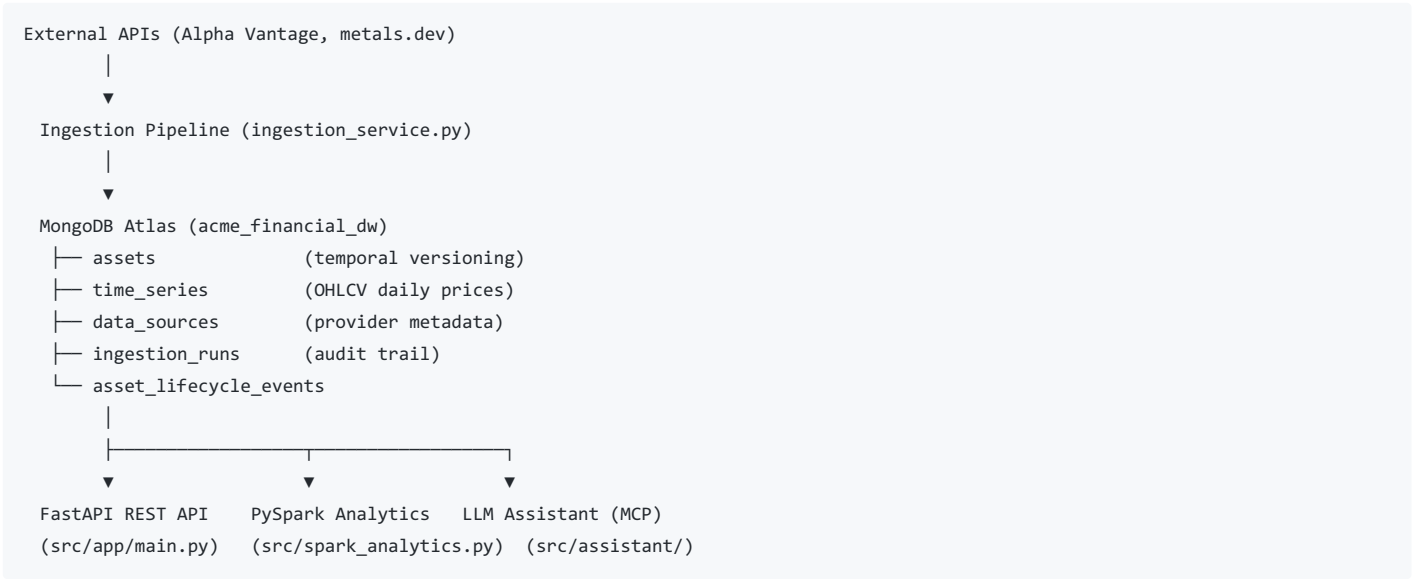
This project implements **Acme Financial DWH**, a MongoDB-based data warehouse for heterogeneous financial instruments. The system ingests market data from external APIs, stores it with temporal versioning and provenance tracking, exposes it through a REST API, runs analytics (including Apache Spark workflows), and provides a grounded LLM assistant via MCP.

The implementation covers four use cases:

Use case	Description	Status
UC1	Data ingestion from external providers	Complete
UC2	REST API for data access (Q1–Q5)	Complete
UC3	Analytics & data mining (REST + Spark)	Complete
UC4	LLM-powered assistant with MCP	Complete

2. What Was Built

2.1 Architecture



Key design choices:

- **MongoDB** as the document store for flexible, heterogeneous financial data.
- **Temporal versioning** on assets (version, valid_from, valid_to, is_active).
- **Provenance** via dataSourceId on every time-series row.
- **Repository layer** (src/repositories.py) to separate API routes from database access.
- **Read-only analytics** — no write operations from API, Spark, or assistant layers.

2.2 UC1 — Data Ingestion

Implemented in ingestion_service.py:

- Multi-provider ETL with retry logic and rate-limit handling.
- Row validation before insert (asset ID, source ID, UTC timestamp, numeric OHLCV).
- Duplicate protection via unique index on (assetId, dataSourceId, timestamp).
- Audit logging to ingestion_runs for every execution.
- Asset lifecycle events recorded in asset_lifecycle_events on version changes.

2.3 UC2 – REST API

Implemented in `src/app/main.py` with FastAPI:

Endpoint	Purpose
<code>GET /assets</code>	List active asset IDs (paginated)
<code>GET /assets/{id}</code>	Latest active asset metadata
<code>GET /data-sources</code>	List registered data sources (paginated)
<code>GET /data-sources/{id}</code>	Full data source details
<code>GET /time-series</code>	Historical prices filtered by asset + source + optional date range
<code>GET /quality/freshness</code>	Data freshness per asset/source pair
<code>GET /quality/summary</code>	Row counts, duplicate risk, latest ingestion status

Interactive documentation: <http://127.0.0.1:8000/docs>

2.4 UC3 – Analytics & Data Mining

REST `analytics` (`src/analytics.py`):

- `GET /analytics/summary` – min/max/average for OHLCV fields.
- `GET /analytics/forecast` – deterministic trend estimate from recent close values.
- `GET /analytics/spark-shape` – flattened rows for Spark/DataFrame ingestion.

Apache Spark `workflows` (`src/spark_analytics.py`):

- Spark aggregation** – grouped metrics (`count`, `min`, `max`, `avg`) by `assetId` and `dataSourceId` using PySpark DataFrames.
- Spark MLlib forecast** – `LinearRegression` with `VectorAssembler` over `timeIndex` to predict next close price, with RMSE and R² metrics.

2.5 UC4 – LLM Assistant via MCP

Implemented in `src/assistant/`:

- 9 read-only DWH tools (assets, data sources, time-series, quality, analytics).
- MCP adapter with environment-based enablement.
- OpenRouter LLM integration for tool planning and answer composition.
- Deterministic fallback when LLM is unavailable.
- Guardrails prevent hallucinated numeric claims — all answers include grounding metadata.

3. Data Used

3.1 Data Sources

Provider	<code>dataSourceId</code>	Instruments	API
Alpha Vantage	<code>alpha_vantage_api_v1</code>	TSLA, AMZN (stocks); BTC, ETH (crypto)	Free tier daily time series
metals.dev	<code>metals_dev_v1</code>	XAG (silver), XPT (platinum)	Spot + timeseries (30-day chunks)

Additional data source records (Bloomberg, CoinMarketCap, NASDAQ Data Link, LBMA, ECB) are registered in `data_sources` for metadata completeness only; ingested time-series data comes from Alpha Vantage and metals.dev.

3.2 MongoDB Collections

Collection	Contents	Approx. size
<code>assets</code>	Asset metadata with temporal versioning	6 active assets

time_series Collection	Daily OHLCV price points Contents	~600 rows Approx. size
data_sources	Provider metadata and API details	7 sources
ingestion_runs	Ingestion audit trail	Per-run documents
asset_lifecycle_events	Version change audit	Per-event documents

3.3 Data Model Highlights

Each time-series document:

```
{
  "assetId": "TSLA",
  "dataSourceId": "alpha_vantage_api_v1",
  "timestamp": "2026-05-08T00:00:00Z",
  "point": {
    "open": 420.1,
    "high": 431.2,
    "low": 418.9,
    "close": 428.35,
    "volume": 1234567
  }
}
```

Each asset document includes temporal fields: `version`, `valid_from`, `valid_to`, `is_active`.

4. How to Reproduce Results

4.1 Prerequisites

- Python 3.11+
- MongoDB Atlas cluster (or local MongoDB)
- Java 17 (for Spark workflows)
- API keys: Alpha Vantage, metals.dev (optional), OpenRouter (optional, for LLM assistant)

4.2 Setup

```
cd /path/to/project
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev]"
```

Create `.env` in the project root (do not commit):

```
MONGO_URI=mongodb+srv://...
MONGO_DB_NAME=acme_financial_dw
ALPHA_VANTAGE_API_KEY=your_key
METAL_API_KEY=your_key           # only for XAG/XPT
ASSISTANT_MCP_ENABLED=true       # for UC4
LLM_API_KEY=your_openrouter_key  # optional
LLM_MODEL=openai/gpt-4o-mini     # optional
```

Initialize database indexes and seed data:

```
python db_setup.py
```

4.3 Ingest Data (UC1)

```
# Default: TSLA + BTC
python ingestion_service.py

# Single symbol
python ingestion_service.py --symbol TSLA
python ingestion_service.py --symbol XAG --force
```

Verify in MongoDB Atlas: collections `assets`, `time_series`, `data_sources`, `ingestion_runs` should be populated.

4.4 Run REST API (UC2)

```
uvicorn app.main:app --app-dir src --reload
```

Open `http://127.0.0.1:8000/docs` and test:

```
curl "http://127.0.0.1:8000/assets"
curl "http://127.0.0.1:8000/assets/TSLA"
curl "http://127.0.0.1:8000/time-series?assetId=TSLA&dataSourceId=alpha_vantage_api_v1"
curl "http://127.0.0.1:8000/analytics/summary?assetId=TSLA&dataSourceId=alpha_vantage_api_v1&startDate=2026-05-01&endDate=2026-05-08"
curl "http://127.0.0.1:8000/quality/freshness"
```

Expected analytics summary for TSLA (May 2026 data):

```
{
  "assetId": "TSLA",
  "count": 6,
  "close": { "min": 389.37, "max": 428.35, "average": 401.93 }
}
```

4.5 Run Spark Analytics (UC3)

Set Java 17:

```
export JAVA_HOME="/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home"
export PATH="$JAVA_HOME/bin:$PATH"
```

Export data from MongoDB (optional – sample file included):

```
.venv/bin/python -m src.spark_analytics export --output data/time_series_export.jsonl
```

Spark aggregation:

```
.venv/bin/python -m src.spark_analytics aggregate \
  --input data/time_series_export.sample.jsonl \
  --output data/spark_aggregations
```

Spark ML forecast:

```
.venv/bin/python -m src.spark_analytics forecast \
  --input data/time_series_export.sample.jsonl \
  --asset-id TSLA \
  --data-source-id alpha_vantage_api_v1
```

Expected forecast output includes `model`: "Spark MLlib LinearRegression", `trainingRows`, `predictedNextClose`, `direction`.

4.6 Test LLM Assistant (UC4)

With MCP and OpenRouter configured in `.env`:

```
curl -X POST "http://127.0.0.1:8000/assistant/query" \
-H "Content-Type: application/json" \
-d '{"question": "What is the latest close price for TSLA from alpha_vantage_api_v1?"}'
```

Expected: "status": "grounded" with query_time_series in grounding metadata.

4.7 Run Automated Tests

```
.venv/bin/python -m pytest          # 75 tests
.venv/bin/python -m ruff check .    # lint
```

5. Project Structure

```
project/
├─ ingestion_service.py      # UC1: multi-provider ETL
├─ db_setup.py              # Database initialization + seed data
├─ quality.py               # Data quality helpers
├─ src/
│   ├─ app/main.py          # UC2: FastAPI REST API
│   ├─ analytics.py         # UC3: REST analytics helpers
│   ├─ spark_analytics.py    # UC3: PySpark aggregation + ML
│   ├─ spark_common.py      # Spark utility helpers
│   ├─ repositories.py      # DAL / repository layer
│   ├─ quality.py           # Quality module (src)
│   └─ assistant/          # UC4: MCP + LLM assistant
│       ├─ service.py
│       ├─ tools.py
│       ├─ mcp_adapter.py
│       └─ models.py
├─ tests/                   # 75 automated tests
├─ data/
│   └─ time_series_export.sample.jsonl
├─ README.md               # Full technical documentation
├─ progress.md             # Development progress tracker
└─ PROJECT_REPORT.md       # This file
```

6. Conclusion

The Acme Financial DWH project delivers a complete data warehouse pipeline: ingestion from multiple providers, temporal versioning, REST API access, analytics (REST and Spark), data quality governance, and a grounded LLM assistant. All components are tested (75 passing tests), documented, and reproducible from the steps above.